MARS ROVER MANIPAL
RESEARCH AI TASKPHASE
Report 2

Adarsh Inaganti
240905294
adarshinaganti006@gmail.com

1. Review questions

1.1. What is the difference between a conference and a journal?

1.1.1 Conference papers

A conference paper is a short, written document submitted for an upcoming conference and later forms the basis for its oral presentation during the conference. While the papers go through a general review to decide on their acceptance, they do not go through a systematic peer review process like journal papers do. All or only selected conference papers may be published in the form of conference proceedings.

1.1.2 Types of conference papers

- ***Abstract:*** An abstract provides a synopsis of your research with a word count of 250 or less. It should contain the central idea, methodology, results, and significance or implications of your study. It should also include the necessary keywords. The abstract needs to be well thought out and presented as it is a primary basis for conference organizers to decide on the acceptance of your paper and for conference participants to assess the relevance of the study to their academic and research interests.
- ***Extended abstract:*** The extended abstract, which may be around two pages long, should briefly describe the gap or problem the study is addressing, discuss relevant literature as it relates to the study, and briefly discuss the methodology employed and any evidence-based results. In addition, it should also mention the title, author names and affiliations, acknowledgments, and references.
- ***Brief or short paper:*** The brief or short paper should be no more than four pages and summarize your research in the most effective way. Be sure to focus on the most important findings, methodology, and significance of the study.
- ***Full research paper:*** This should contain the complete details of your research, numbering around 6 to 8 pages. It should be well structured and consist of the title page, abstract, introduction, methodology, results, significance, acknowledgment, and references.

1.1.3 Journal papers

Journal papers are scientific or academic papers published in journals. All papers submitted to a journal are peer-reviewed to ensure the quality and significance of the study and form the primary basis for its acceptance or rejection. Publishing papers or articles in peer-reviewed and high-impact journals is crucial for the academic and professional growth of researchers and academicians.

1.1.4 Types of journal papers

- ***Original research paper:*** This is the most common type and contains a complete report of the research data. It will be analytical with clear and significant research findings. The paper will be well structured with an abstract, introduction, methodology, results, and discussion sections.
- ***Short report or communication:*** Here, short reports of original research or significant research findings are published because the subject or results may be time-sensitive or the journal editors feel that it will be of considerable interest to the readers.
- ***Review articles*** provide a detailed summary of the literature on a particular research topic. They normally cite around 100 primary research papers.
- ***Case study:*** Case studies describe a significant phenomenon in its actual form without altering the conditions. The main objective is to point out the emergence or occurrence of certain phenomena.
- ***Methods:*** Here, a totally new experimental method, test, or procedure is presented. It can also be an advanced version of an existing method.

1.2 What are workshops and how are they different from conferences and journals?

1.2.1 Workshops

Workshops are more or less informal collections of researchers working on a common topic who meet (often at the same time as a conference) to touch base and discuss the topic. Many conferences started out as workshops and were upgraded to conference status as their popularity grew.

1.2.2 Types of workshop papers

- ***Workshop position paper:*** An informal, often very short, statement of an individual researcher's opinion (position) about a specific issue (often the issue that the workshop deals with). This is the canonical workshop paper, and it exists so that workshop participants can learn of the other participants' opinion before the actual workshop or during a short session at its beginning.
- ***Workshop research paper:*** Some workshops have grown from mere discussion venues into having their own technical program. Research papers submitted to a workshop are often similar to conference papers, although the level of novelty required is generally smaller than for a conference. Most workshops will include presentation slots for research papers.

1.2.3 Differences between workshops, conferences, journals

Aspect	Workshop	Conference	Journal
Size	small	medium/large	not event-based
Focus	narrow	broad (within a field)	broad/deep
Formality	low/medium	high	very high
Speed	fast (weeks/months)	moderate (months)	slow (months-years)
Output	preliminary ideas	established results	complete, archival record

1.3 What is h-index?

1.3.1 Definition

A researcher has an h-index of ***h*** if they have published ***h*** papers, each of which has been cited at least ***h*** times.

1.3.2 Example

Suppose a researcher has 6 papers with the following citation counts:

- ***Paper 1:*** 50 citations
- ***Paper 2:*** 30 citations
- ***Paper 3:*** 20 citations
- ***Paper 4:*** 5 citations
- ***Paper 5:*** 3 citations
- ***Paper 6:*** 1 citation

Here, the ***h-index = 3***, because:

- at least 3 papers have ≥ 3 citations each.
- but not 4 papers with ≥ 4 citations.

1.3.3 Key takeaways

- Higher h-index = consistent, widely cited contributions.
- It's discipline-dependent — fields with lots of papers/citations (like biology or computer science) tend to yield higher h-indices than fields with fewer publications (like philosophy).
- It doesn't account for:
    - A single "super-highly cited" paper (that boosts citations but not h-index much).
    - Author order (first vs last author).
    - Very recent work that hasn't had time to be cited.

1.3.4 Where it's used

- Google Scholar
- Scopus
- Web of Science (WoS)

1.4 What is a citation?

1.4.1 Definition

- In academic and research contexts, a citation is a way of crediting the original source of an idea, method, figure, or wording.
- It usually points to a book, article, paper, dataset, or website that supports, inspires, or is directly quoted in your work.

1.4.2 Why are citations important?

Citations are important in both writing and research because they serve several purposes:

- ***Ethics:*** Give credit to the original author for their work.
- ***Evidence:*** By citing established papers, one can show that their claims are backed by evidence and not just personal opinion.
- ***Depth of research:*** A well-cited paper demonstrates that you have read widely and are aware of prior work.
- ***Academic impact:*** Citations feed into metrics like ***h-index*** and the ***journal impact factor***.

1.5 What are journal quartiles?

1.5.1 Definition

Journal quartiles are a way of ranking academic journals within a subject area based on citation-based metrics *(like Impact Factor or SCImago Journal Rank (SJR)).*

1.5.2 How it works

All journals in a given field are ranked by their impact metric. They are then split into four equal groups (quartiles):

Quartile	Rank range	Meaning
Q1	top 25% of journals	highest ranked in the field
Q2	25%-50%	above average
Q3	50%-75%	below average
Q4	bottom 25%	lowest ranked in the field

1.5.3 Example

Suppose there are 200 journals in Computer Science (AI) ranked by SJR. Journal quartiles are split as:

- ***Q1:*** Journals ranked 1–50
- ***Q2:*** Journals ranked 51–100
- ***Q3:*** Journals ranked 101–150
- ***Q4:*** Journals ranked 151–200

1.5.4 Why it matters

Researchers often prefer publishing in Q1 journals due to a higher prestige, visibility and credibility. Universities and funding agencies sometimes evaluate performance based on quartile rankings.

2. Pandas

2.1 Introduction

2.1.1 Definition

Pandas is a powerful free and open-source Python library designed for data analysis and manipulation. It is built on top of NumPy and is one of the most widely used tools in data science, machine learning, finance, and general-purpose data wrangling.

2.1.2 Use cases

1. Data cleaning and preparation
    - Handle missing values (`dropna()`, `fillna()`).
    - Remove duplicates.
    - Normalize/standardize columns.
    - Rename/reorder columns.
2. Data exploration and analysis
    - Compute summary statistics (`mean()`, `median()`, `describe()`).
    - Filter and query subsets of data.
    - Aggregate with `groupby()`.
    - Spot trends and correlations.
3. Data transformation
    - Merge and join multiple datasets (like SQL `JOIN`).
    - Reshape with `pivot() / melt()`.
    - Apply functions across rows/columns.
4. Input/output (I/O)
    - Read from and write to: CSV, Excel, SQL databases, JSON etc.
    - Fetch remote datasets (via URLs).
5. Time series analysis
    - Parse dates easily (`pd.to_datetime`).
    - Resample (daily → weekly, monthly, etc.).
    - Rolling statistics (e.g., moving averages).
    - Shift and lag operations for forecasting.

2.2 Features

1. ***Data structures***
    - ***Series:*** One-dimensional labeled array
    - ***DataFrame:*** Two-dimensional labeled data structure
2. ***Data handling***
    - Import/export data from CSV, Excel, SQL, JSON, parquet, and more.
    - Handle missing data (drop, fill, interpolate).
    - Merge, join, and concatenate datasets.
3. ***Indexing and selection***
    - Select rows/columns by labels (`.loc`) or positions (`.iloc`).
    - Boolean indexing and conditional filtering.
4. ***Data operations***
    - ***Aggregations:*** `sum()`, `mean()`, `count()`, etc.
    - Grouping with `groupby()` for split-apply-combine operations.
    - Sorting and ranking.
5. ***Time series support***
    - Date parsing, resampling, rolling windows.
    - Very useful in financial and sensor data analysis.

3. NumPy

3.1 Introduction

3.1.1 Definition

NumPy (short for Numerical Python) is one of the most widely used libraries in Python for numerical computing. It provides tools to handle large, multi-dimensional arrays and matrices efficiently, along with a collection of high-performance mathematical functions to operate on them.

3.2 Arrays

A NumPy array is called an `ndarray` (N-dimensional array). Python has a built-in `list` data type, but NumPy arrays are usually considered to be better as they are:

- homogeneous (all elements must be of the same data type).
- stored in contiguous memory (much faster to access and manipulate).
- multidimensional (1D, 2D, 3D, ... ND arrays).

3.2.1 Creating a NumPy array

To perform any operation with NumPy, importing it is required.

```
import numpy as np
```

The `np` keyword will now be used to refer to numpy and all its functions can be accessed by using `np.function()`. NumPy arrays can be created using the function `np.array()`.

3.2.2 1D and 2D arrays

```
1  # 1D array
2  a = np.array([1, 2, 3])
3  a
```

```
array([1, 2, 3])
```

This creates a 1D array `a` with values 1, 2, 3.

```
1  # 2D array
2  b = np.array([[1.0, 2.0, 3.0], [4.0, 5.0, 6.0]])
3  b
```

✓ 0.0s

```
array([[1., 2., 3.],
       [4., 5., 6.]])
```

This creates a 2x3 2D array `b`.

Shape
The attribute `shape` returns a `tuple` in the form `(rows, columns)` of an array.

```
1  # shape
2  print(f"a: {a.shape}")
3  print(f"b: {b.shape}")
```

✓ 0.0s

```
a: (3,)
b: (2, 3)
```

The array `a` has 3 elements, thus its shape is `(3,)`. The array `b` is a 2x3 array, thus its shape is `(2, 3)`.

Data type
The attribute `dtype` returns the data type of the elements of the array.

```
1  # type
2  print(f"a: {a.dtype}")
3  print(f"b: {b.dtype}")
```
✓ 0.0s
```
a: int64
b: float64
```

The array `a` is an array of integers and each integer occupies 8 bytes (64 bits). Thus, its type is `int64`. Whereas, the array `b` is an array of floating point values and each floating point value occupies 8 bytes (64 bits). Thus, its type is `float64`.

Item size
The attribute `itemsize` returns the size (in bytes) of each element in the array.

```
1  # item size
2  print(f"a: {a.itemsize} bytes")
3  print(f"b: {b.itemsize} bytes")
```
✓ 0.0s
```
a: 8 bytes
b: 8 bytes
```

We have seen that the `dtype` of `a` is `int64`, implying each element occupies 8 bytes. Similarly, the `dtype` of `b` is `float64`, where each element also occupies 8 bytes.

Total size
The attribute `totalsize` returns the total size of the array (in bytes). Total size can also be calculated by multiplying the number of elements of the array and the item size.

```
1  # total size
2  print(f"a: {a.nbytes} bytes")
3  print(f"b: {b.nbytes} bytes")
```
✓ 0.0s
```
a: 24 bytes
b: 48 bytes
```

Total size of `a` = Number of elements * item size = 3 * 8 = 24 bytes
Total size of `b` = Number of elements * item size = 6 * 8 = 48 bytes

Accessing and modifying data

For this example, let us consider a 2x7 2D array `a` with the values `[1, 2, …, 7]`, `[8, 9, …, 14]`.

```
1  a = np.array([[1, 2, 3, 4, 5, 6, 7], [8, 9, 10, 11, 12, 13, 14]])
2  a
```

✓ 0.0s

```
array([[ 1,  2,  3,  4,  5,  6,  7],
       [ 8,  9, 10, 11, 12, 13, 14]])
```

Accessing a specific element

A specific element can be accessed using square brackets.

For example, if we want to access the element in the 2nd row and the 6th column (`13`), we use `a[1, 5]`. This is because indexing starts from `0` instead of `1`, which means that the first element is denoted by `0`, second by `1` and so on.

Negative integers can also be used to count backwards. For example, if we want to access the element in the 1st row and the 2nd last column (`6`), we use `a[0, -2]`.

```
1  # specific element [r, c]
2  print(a[1, 5]) # row 2, column 6
3  print(a[0, -2]) # row 1, 2nd last column
```

✓ 0.0s

```
13
6
```

Accessing a specific row

The `:` operator can be used to select a group of elements. For example, `a[0, :]` can be used to access the first row, and `a[1, :]` can be used for the second row.

```
1  # specific row
2  print(a[0, :]) # first row
3  print(a[1, :]) # second row
```

```
[1 2 3 4 5 6 7]
[ 8  9 10 11 12 13 14]
```

Accessing a specific column

The `:` operator can also be used for columns. Using `a[:, 2]`, we can access the third column and using `a[:, -1]` we can access the last column.

```
# specific column
print(a[:, 2]) # third column
print(a[:, -1]) # last column
```

```
[ 3 10]
[ 7 14]
```

3.2.3 3D arrays

3D arrays can be created like so:

```
b = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
b
```

```
array([[[1, 2],
        [3, 4]],

       [[5, 6],
        [7, 8]]])
```

Rules for accessing and modifying remain the same for any N-dimensional array.

3.2.4 Special arrays

To create an array of all zeroes, we can use `np.zeros(shape)`.

```
# all zeros
np.zeros((2, 3))
```

```
array([[0., 0., 0.],
       [0., 0., 0.]])
```

This can also be done for an array with all ones using `np.ones(shape)`.

```
# all ones
np.ones((4, 2, 2))
```

```
array([[[1., 1.],
        [1., 1.]],

       [[1., 1.],
        [1., 1.]],

       [[1., 1.],
        [1., 1.]],

       [[1., 1.],
        [1., 1.]]])
```

For any specific number, we can use `np.full(shape, number)`.

```
# any number
np.full((2, 2), 99)
```

```
array([[99, 99],
       [99, 99]])
```

We can pass the shape of another array and make a new array by using `np.full_like(shape, number)`.

```
# full_like
np.full_like(a, 4)
```

```
array([[4, 4, 4, 4, 4, 4, 4],
       [4, 4, 4, 4, 4, 4, 4]])
```

Random arrays can be made using `np.random.rand(shape)`.

```
# random decimal numbers
np.random.rand(4, 2)
```

```
array([[0.26115772, 0.56376272],
       [0.92992973, 0.04611488],
       [0.97708063, 0.28275647],
       [0.5254579 , 0.98949668]])
```

To use another array's shape for a random array, `np.random_sample(shape)` is used.

```
# passing a shape
np.random.random_sample(a.shape)
```

```
array([[0.28553136, 0.1117217 , 0.51739121, 0.52764064, 0.51841011,
        0.29218433, 0.20245467],
       [0.28085639, 0.31225435, 0.84747995, 0.8562077 , 0.29233416,
        0.22028743, 0.29275139]])
```

For an array of random integers, we use `np.random.randint(limit, shape)`.

```
# random integers
np.random.randint(7, size=(3, 3))
```

```
array([[0, 5, 1],
       [5, 4, 6],
       [5, 4, 0]])
```

An identity matrix of order n can be made using `np.identity(n)`.

```
# identity matrix
np.identity(3)
```

```
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])
```

3.2.5 Mathematical operations on arrays

Let us consider an array `a` with values `[1, 2, 3, 4]`.

```
a = np.array([1, 2, 3, 4])
a
```

```
array([1, 2, 3, 4])
```

Arithmetic operations can be performed by using arithmetic operators. This changes every element of the array.

```
a + 2
```

```
array([3, 4, 5, 6])
```

```
a - 2
```

```
array([-1,  0,  1,  2])
```

```
a * 2
```

```
array([2, 4, 6, 8])
```

```
a / 2
```

```
array([0.5, 1. , 1.5, 2. ])
```

Two arrays can also be added or subtracted if they are of the same shape.

```
1  b = np.array([0, 1, 1, 0])
2  a + b
✓ 0.0s
array([1, 3, 4, 4])
```

Trigonometric functions can also be performed by using `np.sin()`, `np.cos()` etc.

```
1  # trigonometric functions
2  print(f"sin: {np.sin(a)}")
3  print(f"cos: {np.cos(a)}")
4  print(f"tan: {np.tan(a)}")
✓ 0.0s
sin: [ 0.84147098  0.90929743  0.14112001 -0.7568025 ]
cos: [ 0.54030231 -0.41614684 -0.9899925  -0.65364362]
tan: [ 1.55740772 -2.18503986 -0.14254654  1.15782128]
```

4. Git

4.1 Introduction

4.1.1 What is git?

Git is a distributed version control system (VCS) used to track changes in files, especially source code, during software development. It helps multiple people work on a project at the same time without overwriting each other's work and keeps a full history of all changes so you can go back if needed.

4.1.2 Where is git used?

Git is used anywhere version control is needed, but it is especially popular in software development. Popular platforms like GitHub and GitLab are built on Git.

4.2 Workflow

The steps of the git workflow are as follows:

1. **Initializing:** A local directory is initialized and git VCS is now active for the specific project.
2. **Staging:** New files are created and they are added or staged to git. This lets git start tracking those files for changes.
3. **Committing:** Once the changes are made, a commit is created with a message detailing the changes in the specific file(s).
4. **Pushing:** The changes are pushed to a remote branch on a remote repository on a platform like GitHub where the repository is made.
5. **Merging:** The changes can be compared and merged to another branch as per the use case of the project.

4.3 Common commands

4.3.1 Init

Used to initialize the git repository.

```
$ cd <path-to-directory>
$ git init
```

4.3.2 Remote add

Used to add an alias to the remote repository we are working with. The most commonly used alias is `origin`. After this alias is added, we can refer to this remote repository from the command line by using its alias.

```
$ git remote add <remote> <url>
```

4.3.3 Add

Used to stage changes.

- `git add .` is used to add all new files.
- `git add <filename>` is used to add one file.
- `git add <file1>, <file2>, . . .` is used to add multiple files.

4.3.4 Commit

Used to commit the changes.

```
$ git commit -m "commit message"
```

4.3.5 Push

Used to push to a remote repository.

```
$ git push -u <branch> <remote>
```

4.3.6 Fetch

Used to get new changes from the remote repository without merging them locally.

```
$ git fetch <remote-name>
```

4.3.7 Pull

Used to get new changes from the remote repository and immediately merge them into the existing local branch.

```
$ git pull <remote> <branch>
```

4.3.8 Clone

Used to download a remote repository locally.

```
$ git clone <url>
```

4.4 Branching

Branching is a very useful feature of git. It allows multiple people to work on the same project, or for new features to be tested before they are published to the main branch.

To create a new branch, we can use the `git checkout` command.

```
$ git checkout -b <branch>
```

This tells git that we are working on the new branch. All commits and pushes made from now on will go only to the new branch. After testing is done, this branch can be merged to the `main` branch. For example, let us consider a branch `test` which is to be merged to the `main` branch. To merge these branches, the commands are as follows:

```
$ git checkout main       # ensure that we are on main
$ git pull origin main    # pull all changes from remote
$ git checkout test       # switch to test branch
$ git merge main          # merge test with main
```

We can also merge two branches from GitHub's website by using pull requests.

4.5 Undoing

One of the biggest advantages of VCS is that it allows us to roll back or undo changes. This is particularly helpful if a commit is causing problems.
To reset by one commit and keep changes, we can do a soft reset.

```
$ git checkout <branch>       # to ensure correct branch
$ git reset --soft HEAD~1     # soft reset by one commit
```

To reset by one commit and discard changes, we can do a hard reset.

```
$ git checkout <branch>       # to ensure correct branch
$ git reset --hard HEAD~1     # hard reset by one commit
```

Every commit has a specific hash associated with it. To revert to a specific commit, we can use `git revert`.

```
$ git revert <commit-hash>
```

4.6 Forking

Forking in Git (and platforms like GitHub/GitLab) is when you make your own copy of someone else's repository under your account. Forking is different from cloning, because a fork lives independently on your remote account, not just on your local machine.

4.6.1 Why is forking used?

1. Independent copy:
    - You can modify, experiment, and commit freely without affecting the original repository.
2. Used for collaboration:
    - Common in open-source projects. You fork a repo, make changes, then propose them back via a pull request.
3. Keeps original repo intact:
    - The owner's repository is not affected until they accept your changes.

4.6.2 Forking workflow

First, a repository should be forked on GitHub. This can be done by visiting the repository you would like to fork and clicking on the fork button. Now, this repository gets registered under your username and we can work on it by cloning it first.

```
$ git clone https://github.com/your-username/forked-repo.git
```

Changes can now be made locally and pushed.

```
$ git push origin main
```

To merge it into the original repository, pull requests can be created.

5. References

- https://researcher.life/blog/article/conference-paper-vs-journal-paper-whats-the-difference/
- https://niklaselmqvist.medium.com/workshops-conferences-journals-oh-my-ed91e206007a
- https://arxiv.org/abs/1805.03522
- https://arxiv.org/abs/1806.10674
- https://en.wikipedia.org/wiki/Academic_conference
- https://en.wikipedia.org/wiki/Conference_proceedings
- https://inomics.com/blog/whats-the-difference-between-a-conference-a-seminar-a-workshop-and-a-symposium-1075915
- https://pubscholars.org/conference/conference-papers-vs-journal-paper-major-difference/
- https://spacestudies.co.uk/blog/difference-between-a-conference-paper-and-a-journal-paper/
- https://libguides.bodleian.ox.ac.uk/maths/journals-and-conference
- https://www.pnas.org/doi/10.1073/pnas.0507655102
- https://beckerguides.wustl.edu/authors/hindex
- https://pmc.ncbi.nlm.nih.gov/articles/PMC10025721/
- https://pmc.ncbi.nlm.nih.gov/articles/PMC10771139/

- https://library.cumc.columbia.edu/kb/author-impact-metric-h-index
- https://arxiv.org/abs/1108.3901
- https://arxiv.org/html/2503.13456
- https://arxiv.org/abs/2102.03234
- https://pmc.ncbi.nlm.nih.gov/articles/PMC5953266/
- https://pmc.ncbi.nlm.nih.gov/articles/PMC8712974/
- https://guides.lib.uw.edu/research/citations/citationwhat
- https://nps.edu/web/gwc/why-cite-a-writer-s-perspective
- https://www.pcc.edu/library/research/reasons-for-citing-sources/
- https://web.grinnell.edu/Dean/Tutorial/EUS/IC.pdf
- https://support.clarivate.com/ScientificandAcademicResearch/s/article/Journal-Citation-Reports-Quartile-rankings-and-other-metrics
- https://www.scimagojr.com/help.php
- https://www.editage.us/blog/guide-to-journal-rankings-what-are-quartiles-q1-q2-q3-q4-journal/
- https://www.enago.com/academy/q1-to-q4-understanding-journal-quartiles/
- https://blog.wos-journal.info/what-is-quartile-ranking/
- https://mbru.libguides.com/c.php?g=716266&p=5218231
- https://www.manuscriptedit.com/scholar-hangout/quartiles-of-the-journals-and-the-secret-of-publishing/
- https://arxiv.org/abs/2005.02726
- https://www.kaggle.com/learn/pandas
- https://youtu.be/QUT1VHiLmmI
- https://youtu.be/RGOj5yH7evk
- https://learngitbranching.js.org/